

Relational Calculus

Dr. Gunjan Chugh
Assistant Professor
DTU

Relational Calculus

- Relational Calculus is a **Non-Procedural Query Language**.
- **Meaning of Non-Procedural:**
 - You do **not** write step-by-step instructions
 - You only specify the **conditions** for the required data
 - The DBMS itself decides the best way to fetch the result
- There are **two types**:
 - **Tuple Relational Calculus (TRC)**
 - **Domain Relational Calculus (DRC)**

Example:

- If you say:
 “I want students from the CSE department”
- You are not explaining:
 - first scan the table
 - then apply filtering
 - then produce output
- You are only giving the **condition**, and DBMS handles the execution.

Tuple Relational Calculus (TRC)

- **Non-procedural query language**
- Describes WHAT data is needed, not HOW to get it
- Based on first-order predicate logic
- **Uses tuple variables (represents rows)**

Tuple Relational Calculus

- Tuple Relational Calculus (TRC) is a **non-procedural query language** used to retrieve data from relational databases by **describing the properties of the required data** (not how to fetch it).
- It is **based on first-order predicate logic** and uses **tuple variables** to represent rows of tables.

Syntax:

$\{ t \mid P(t) \}$

t = tuple variable

P(t) = predicate condition

TRC

Logical Operators in TRC:

- $\wedge$: AND
- $\vee$: OR
- $\neg$: NOT

Quantifiers:

- $\exists t \in r (Q(t)) \rightarrow$ There exists a tuple t in relation r satisfying predicate $Q(t)$
- $\forall t \in r (Q(t)) \rightarrow$ For all tuples t in relation r , predicate $Q(t)$ holds

TRC Example 1

Find students in CSE department

$$\{ t \mid t \in \text{STUDENT} \wedge t.\text{Dept} = \text{'CSE'} \}$$

TRC Example 2

Find names of students with marks > 80

$$\{ t.\text{Name} \mid t \in \text{STUDENT} \wedge t.\text{Marks} > 80 \}$$

TRC Example 3

- Find employees earning salary > 50000

$\{ t \mid t \in \text{EMPLOYEE} \wedge t.\text{Salary} > 50000 \}$

TRC Examples

Example 4: Find students with Roll > 2

- $\{ t \mid t \in \text{STUDENT} \wedge t.\text{Roll} > 2 \}$

Example 5: Find employees working in HR department

- $\{ t.\text{Name} \mid t \in \text{EMPLOYEE} \wedge t.\text{Dept} = \text{'HR'} \}$

Example 6: TRC

branch (branch_name, branch_city, assets)
customer (customer_name, customer_street, customer_city)
account (account_number, branch_name, balance)
loan (loan_number, branch_name, amount)
depositor (customer_name, account_number)
borrower (customer_name, loan_number)

Query: Find the loan-number, branch-name and amount for all loans of over > \$2000.

Sol: $\{t \mid t \in \text{loan} \wedge t[\text{amount}] > 2000\}$

Query: Find the loan number for each loan of an amount greater than 2000

Sol: $\{t \mid \exists s \in \text{loan} (t[\text{loan-number}] = s[\text{loan-number}] \wedge s[\text{amount}] > 2000)\}$

Example 7: TRC

branch (*branch_name*, *branch_city*, *assets*)
customer (*customer_name*, *customer_street*, *customer_city*)
account (*account_number*, *branch_name*, *balance*)
loan (*loan_number*, *branch_name*, *amount*)
depositor (*customer_name*, *account_number*)
borrower (*customer_name*, *loan_number*)

Query: Find the names of all customers who are having a loan at the PERRYRIDGE branch.

Sol: $\{t \mid \exists s \in \text{borrower } (t[\text{customer-name}] = s[\text{customer-name}]$
 $\wedge \exists u \in \text{loan } (u[\text{loan-number}] = s[\text{loan-number}]$
 $\wedge u[\text{branch-name}] = \text{"Perryridge"}) \}$

Query: Find the names of all customers having a loan, an account or both at the bank.

Sol: $\{t \mid \exists s \in \text{borrower } (t[\text{customer-name}] = s[\text{customer-name}])$
 $\vee \exists u \in \text{depositor } (t[\text{customer-name}] = u[\text{customer-name}]) \}$

Example 8: TRC

branch (branch_name, branch_city, assets)
customer (customer_name, customer_street, customer_city)
account (account_number, branch_name, balance)
loan (loan_number, branch_name, amount)
depositor (customer_name, account_number)
borrower (customer_name, loan_number)

Query: Find the names of all customers having a loan and an account at the bank.

Sol: $\{t \mid \exists s \in \text{borrower } (t[\text{customer-name}] = s[\text{customer-name}])$
 $\wedge \exists u \in \text{depositor } (t[\text{customer-name}] = u[\text{customer-name}]) \}$

Domain Relational Calculus (DRC)

- Non-procedural query language
- **Uses domain variables (attribute values)**
- Based on predicate logic
- It describes queries by specifying a set of conditions or formulas that the data must satisfy.
- These conditions are written using domain variables and predicates, and it returns a relation that satisfies the specified conditions.

DRC Syntax

- A general form of a DRC query is written as:

$$\{ \langle x_1, x_2, x_3, \dots, x_n \rangle \mid P(x_1, x_2, x_3, \dots, x_n) \}$$

where,

$\langle x_1, x_2, x_3, \dots, x_n \rangle$ represents resulting domains variables and

$P(x_1, x_2, x_3, \dots, x_n)$ represents the condition or formula equivalent to the Predicate calculus.

Table 1: Customer

Customer Name	Street	City
Debomit	Kadamtala	Alipurduar
Sayantan	Udaypur	Balurghat
Soumya	Nutanchati	Bankura
Ritu	Juhu	Mumbai

Table 2: Loan

Loan Number	Branch Name	Amount
L01	Main	200
L03	Main	150
L10	Sub	90
L08	Main	60

Table 3: Borrower

Customer Name	Loan Number
Ritu	L01
Debomit	L08
Soumya	L03

DRC Example 1

- Find loans with amount ≥ 100

$$\{ \langle l, b, a \rangle \mid \langle l, b, a \rangle \in \text{Loan} \wedge a \geq 100 \}$$

Table 1: Customer

Customer Name	Street	City
Debomit	Kadamtala	Alipurduar
Sayantan	Udaypur	Balurghat
Soumya	Nutanchati	Bankura
Ritu	Juhu	Mumbai

Table 2: Loan

Loan Number	Branch Name	Amount
L01	Main	200
L03	Main	150
L10	Sub	90
L08	Main	60

Table 3: Borrower

Customer Name	Loan Number
Ritu	L01
Debomit	L08
Soumya	L03

DRC Example 2

- Find loan IDs with amount ≥ 150

$\{ \langle l \rangle \mid \exists b, a (\langle l, b, a \rangle \in \text{Loan} \wedge a \geq 150) \}$

Table 1: Customer

Customer Name	Street	City
Debomit	Kadamtala	Alipurduar
Sayantan	Udaypur	Balurghat
Soumya	Nutanchati	Bankura
Ritu	Juhu	Mumbai

Table 2: Loan

Loan Number	Branch Name	Amount
L01	Main	200
L03	Main	150
L10	Sub	90
L08	Main	60

Table 3: Borrower

Customer Name	Loan Number
Ritu	L01
Debomit	L08
Soumya	L03

DRC Example 3

- Find customers who borrowed loans from Main branch

$\{ \langle c, a \rangle \mid \exists l (\langle c, l \rangle \in \text{borrower} \wedge \exists b (\langle l, b, a \rangle \in \text{loan} \wedge (b = \text{"Main"}))) \}$

STUDENT Table

Roll No (r)	Name (n)	Department (d)
101	Amit	CSE
102	Priya	ECE
103	Rahul	ME
104	Neha	ECE
105	Karan	Civil

EMPLOYEE Table

Emp ID (e)	Name (n)	Salary (s)
E01	Rajesh	55000
E02	Simran	35000
E03	Mohit	28000
E04	Anjali	45000
E05	Vikram	39000

DRC Examples

Example 4: Find names of students in ECE

- $\{ \langle n \rangle \mid \exists r, d (\langle r, n, d \rangle \in \text{STUDENT} \wedge d = \text{'ECE'}) \}$

Example 5: Find employee IDs with salary < 40000

- $\{ \langle e \rangle \mid \exists n, s (\langle e, n, s \rangle \in \text{EMPLOYEE} \wedge s < 40000) \}$

Example 6: DRC

branch (branch_name, branch_city, assets)

customer (customer_name, customer_street, customer_city)

account (account_number, branch_name, balance)

loan (loan_number, branch_name, amount)

depositor (customer_name, account_number)

borrower (customer_name, loan_number)

Query: Find the loan-number, branch-name and amount for all loans of over > \$2000.

Sol: $\{ \langle l, b, a \rangle \mid \langle l, b, a \rangle \in loan \wedge a > 1200 \}$

Query: Find the names of all customers who have a loan of over \$1200

Sol: $\{ \langle c \rangle \mid \exists l, b, a (\langle c, l \rangle \in borrower \wedge \langle l, b, a \rangle \in loan \wedge a > 1200) \}$

Example 7: DRC

branch (branch_name, branch_city, assets)

customer (customer_name, customer_street, customer_city)

account (account_number, branch_name, balance)

loan (loan_number, branch_name, amount)

depositor (customer_name, account_number)

borrower (customer_name, loan_number)

Query: Find the names of all customers who have a loan from the Perryridge branch and find the loan amount.

Sol:

$\{ \langle c, a \rangle \mid \exists l (\langle c, l \rangle \in \text{borrower} \wedge \exists b (\langle l, b, a \rangle \in \text{loan} \wedge b = \text{"Perryridge"})) \}$

or

$\{ \langle c, a \rangle \mid \exists l (\langle c, l \rangle \in \text{borrower} \wedge \langle l, \text{"Perryridge"}, a \rangle \in \text{loan}) \}$

Example 8: DRC

branch (*branch_name*, *branch_city*, *assets*)
customer (*customer_name*, *customer_street*, *customer_city*)
account (*account_number*, *branch_name*, *balance*)
loan (*loan_number*, *branch_name*, *amount*)
depositor (*customer_name*, *account_number*)
borrower (*customer_name*, *loan_number*)

Query: Find the names of all customers having a loan, an account, or both at the Perryridge branch.

Sol: $\{ \langle c \rangle \mid \exists l (\langle c, l \rangle \in \text{borrower} \wedge \exists b, a (\langle l, b, a \rangle \in \text{loan} \wedge b = \text{"Perryridge"})) \vee \exists a (\langle c, a \rangle \in \text{depositor} \wedge \exists b, n (\langle a, b, n \rangle \in \text{account} \wedge b = \text{"Perryridge"}))) \}$

TRC vs DRC

Tuple Relational Calculus (TRC)	Domain Relational Calculus (DRC)
Tuple variables (represent complete rows)	Domain variables (represent attribute values)
Entire tuples (records)	Individual attribute values (columns)
$\{ t \mid P(t) \}$	$\{ \langle x_1, x_2, \dots \rangle \mid P(x_1, x_2, \dots) \}$
Row-based selection	Value-based selection
t.Name, t.Dept	$\langle n, d \rangle$ style variables
Easier and more intuitive	More abstract and complex
Mainly theoretical, foundation of SQL	Mainly theoretical, logic-based queries
Uses predicates + quantifiers	Uses predicates + quantifiers
Set of tuples (rows)	Set of attribute values

Comparison

SQL	Relational Algebra (RA)	Relational Calculus (RC)
Declarative language	Procedural language	Declarative language
What data is needed	How to get the data step-by-step	What data is needed
Practical (used in real DBMS)	Theoretical + operational	Theoretical + logical
Mix of algebra + calculus concepts	Set operations (σ , π , $\bowtie$, $\cup$, $-$)	Predicate logic + quantifiers
Directly executed in DBMS	Used internally for query processing	Used as foundation for query languages
SELECT-FROM-WHERE	Operators like selection, projection, join	Logical predicates and conditions
User-friendly querying	Stepwise data manipulation	Description of required result
SELECT Name FROM Student WHERE Dept='CSE';	$\sigma_{\text{Dept}='CSE'}(\text{Student})$	$\{ t \mid t \in \text{Student} \wedge t.\text{Dept}='CSE' \}$

Practice Questions (TRC & DRC)

1. Write a TRC query to find students whose Dept = 'CSE'.
2. Write a TRC query to find employees with Salary > 60000.
3. Write a TRC query to find names of students with Marks < 50.
4. Write a DRC query to find loan IDs with Amount > 200.
5. Write a DRC query to find customer names who borrowed from branch 'Rohini'.